Работа с файлами в UI-автотестах

Скачивание PDF и Excel + загрузка файлов через Selenide

Java AQA • Selenide • JUnit 5

Практический урок

1. Цель урока

На уроке разбираем типичный сценарий UI-автоматизации: тест скачивает файл с тестового сайта, проверяет содержимое файла, а затем отдельно учимся загружать файл обратно через веб-интерфейс.

После урока ученик должен уметь

- скачать файл через Selenide и получить объект java.io.File;
- проверить, что файл действительно был получен;
- прочитать PDF и проверить текст или количество страниц;
- прочитать Excel и получить данные из листа или конкретной ячейки;
- загрузить локальный файл через input[type=file];
- разнести работу с файлами по Page Object, тесту и отдельным Utils-классам.

> **Главная идея**
>
> UI-тест отвечает за пользовательский сценарий, Page Object - за взаимодействие со страницей, а PdfUtils / ExcelUtils - за чтение и проверку содержимого скачанных файлов.

Логика занятия

Этап	Что делаем
1	Авторизуемся на тестовом сайте
2	Переходим в раздел документов
3	Скачиваем PDF через Selenide
4	Проверяем PDF через pdf-test
5	Скачиваем / читаем Excel через xls-test + Apache POI
6	Разбираем загрузку файла на примере смены аватарки

2. Скачивание файла через Selenide

Метод download() у SelenideElement возвращает java.io.File. Этот объект дальше можно передать в утилиты для чтения PDF, Excel или других форматов.

DownloadTest.java

```
import com.codeborne.selenide.Selenide;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import pages.DocumentPage;
import pages.LoginPage;

import java.io.File;

public class DownloadTest extends BaseTest {

    @Test
    public void test() {
        Selenide.open("", LoginPage.class)
                .setLogin("admin")
```

```
                .setPassword("admin")
                .clickLoginButton();

        File file = Selenide.open("/#Document", DocumentPage.class)
                .downloadPdfFile();

        Assertions.assertNotNull(file);
    }
}
```

Что здесь важно

- Авторизация выполняется через LoginPage.
- После перехода на страницу документов Page Object скачивает PDF.
- Результат downloadPdfFile() - обычный File.
- assertNotNull(file) - минимальная проверка: объект файла был возвращен.

> **Практика**
> После базовой проверки полезно проверить file.exists(), file.length() > 0, имя файла или его содержимое. Главная проверка должна соответствовать бизнес-смыслу теста.

3. Page Object для скачивания PDF

DocumentPage.java

```
package pages;

import com.codeborne.selenide.SelenideElement;
import io.qameta.allure.Step;

import java.io.File;

import static com.codeborne.selenide.Condition.visible;
import static com.codeborne.selenide.Selenide.$x;

public class DocumentPage {

    private final SelenideElement downloadFile =
            $x("//td[@data-name='file']//a[contains(@title, '.pdf')]");

    @Step("Скачать PDF-файл")
    public File downloadPdfFile() {
        return downloadFile.shouldBe(visible).download();
    }
}
```

Разбор метода

Фрагмент	Назначение
shouldBe(visible)	Перед скачиванием убеждаемся, что ссылка отображается.
download()	Скачивает файл и возвращает java.io.File.
@Step	Делает действие понятным в Allure-отчете.
File	Передается дальше в PdfUtils или ExcelUtils.

4. Проверка PDF

Maven-зависимость

pom.xml

```
<dependency>
    <groupId>com.codeborne</groupId>
    <artifactId>pdf-test</artifactId>
    <version>2.1.1</version>
    <scope>test</scope>
</dependency>
```

Библиотека pdf-test позволяет открыть PDF в тесте и получить текст, количество страниц и другие данные, которые затем можно использовать в assertions.

PdfUtils

PdfUtils.java

```
package utils;

import com.codeborne.pdftest.PDF;

import java.io.File;
import java.io.IOException;
import java.util.Locale;

public final class PdfUtils {

    private PdfUtils() {
    }

    /** Получить весь текст из PDF. */
    public static String getText(File file) {
        return load(file).text;
    }

    /** Получить текст конкретной страницы. Нумерация с 1. */
    public static String getPageText(File file, int pageNumber) {
        PDF pdf = load(file);

        if (pageNumber < 1 || pageNumber > pdf.numberOfPages) {
            throw new IllegalArgumentException(
                    "Страницы " + pageNumber + " нет в PDF. " +
                    "Всего страниц: " + pdf.numberOfPages
            );
        }

        try {
            return new PDF(file, pageNumber, pageNumber).text;
        } catch (IOException e) {
            throw new RuntimeException(
                    "Не удалось прочитать PDF: " + file.getAbsolutePath(), e
            );
        }
    }

    /** Получить количество страниц. */
    public static int getPageCount(File file) {
        return load(file).numberOfPages;
    }

    /** Проверить, содержится ли текст в PDF. */
    public static boolean containsText(File file, String expectedText) {
        return getText(file).contains(expectedText);
    }

    /** Проверить текст без учета регистра. */
    public static boolean containsTextIgnoreCase(File file, String expectedText) {
        return getText(file)
                .toLowerCase(Locale.ROOT)
                .contains(expectedText.toLowerCase(Locale.ROOT));
    }

    /** Проверить, что PDF не пустой. */
    public static boolean isNotEmpty(File file) {
        return !getText(file).isBlank();
```

```
    }

    private static PDF load(File file) {
        try {
            return new PDF(file);
        } catch (IOException e) {
            throw new RuntimeException(
                    "Не удалось прочитать PDF: " + file.getAbsolutePath(), e
            );
        }
    }
}
```

Что умеет PdfUtils

Метод	Результат
getText(file)	Весь текст PDF.
getPageText(file, 2)	Текст только второй страницы.
getPageCount(file)	Количество страниц.
containsText(...)	Поиск текста с учетом регистра.
containsTextIgnoreCase(...)	Поиск без учета регистра.
isNotEmpty(file)	Проверка, что извлеченный текст не пустой.

Примеры проверок PDF

Примеры assertions

```
Assertions.assertTrue(file.exists());
Assertions.assertTrue(file.length() > 0);

Assertions.assertTrue(PdfUtils.isNotEmpty(file));
Assertions.assertTrue(PdfUtils.containsText(file, "Invoice"));
Assertions.assertEquals(2, PdfUtils.getPageCount(file));
```

> **Важно**
> Не нужно проверять все подряд. Проверяйте те свойства PDF, которые доказывают, что пользователь скачал правильный документ: ожидаемый текст, номер документа, дату, количество страниц и т. п.

5. Проверка Excel

Maven-зависимость

pom.xml

```
<dependency>
    <groupId>com.codeborne</groupId>
    <artifactId>xls-test</artifactId>
    <version>1.8.0</version>
    <scope>test</scope>
</dependency>
```

В ExcelUtils используется XLS из xls-test и классы Apache POI: Workbook, Sheet, Row, Cell и DataFormatter.

ExcelUtils

ExcelUtils.java

```
package utils;

import com.codeborne.xlstest.XLS;
import org.apache.poi.ss.usermodel.*;

import java.io.File;

public final class ExcelUtils {

    private ExcelUtils() {
    }

    /** Прочитать Excel-файл. */
    public static XLS read(File file) {
        return new XLS(file);
    }

    /** Получить весь текст из Excel-файла. */
    public static String getText(File file) {
        Workbook workbook = read(file).excel;
        DataFormatter formatter = new DataFormatter();
        StringBuilder result = new StringBuilder();

        for (Sheet sheet : workbook) {
            for (Row row : sheet) {
                for (Cell cell : row) {
                    String value = formatter.formatCellValue(cell);
                    if (!value.isBlank()) {
                        result.append(value).append(" ");
                    }
                }
                result.append(System.lineSeparator());
            }
        }
        return result.toString().trim();
    }

    /** Получить текст конкретного листа. */
    public static String getSheetText(File file, String sheetName) {
        Sheet sheet = getSheet(file, sheetName);
        DataFormatter formatter = new DataFormatter();
        StringBuilder result = new StringBuilder();

        for (Row row : sheet) {
            for (Cell cell : row) {
                String value = formatter.formatCellValue(cell);
                if (!value.isBlank()) {
                    result.append(value).append(" ");
                }
            }
            result.append(System.lineSeparator());
        }
        return result.toString().trim();
    }

    /** Нумерация строки и столбца начинается с 0. */
    public static String getCellValue(
            File file, String sheetName, int rowNumber, int columnNumber) {

        Sheet sheet = getSheet(file, sheetName);
        Row row = sheet.getRow(rowNumber);

        if (row == null) {
            throw new IllegalArgumentException(
                    "Строка " + rowNumber + " не найдена"
            );
        }

        Cell cell = row.getCell(columnNumber);
        if (cell == null) {
            return "";
        }

        return new DataFormatter().formatCellValue(cell);
    }

    /** Получить количество листов. */
    public static int getSheetCount(File file) {
        return read(file).excel.getNumberOfSheets();
    }

    /** Получить лист. */
```

```
    private static Sheet getSheet(File file, String sheetName) {
        Sheet sheet = read(file).excel.getSheet(sheetName);

        if (sheet == null) {
            throw new IllegalArgumentException(
                    "Лист '" + sheetName + "' не найден"
            );
        }
        return sheet;
    }
}
```

Основные методы ExcelUtils

Метод	Что возвращает
read(file)	Объект XLS.
getText(file)	Текст всех заполненных ячеек всех листов.
getSheetText(file, "Users")	Текст только указанного листа.
getCellValue(file, "Users", 1, 2)	Значение ячейки: строка 1, столбец 2.
getSheetCount(file)	Количество листов в книге.

> **Индексы Excel**
> В getCellValue() строки и столбцы считаются с 0. Например, rowNumber = 0 и columnNumber = 0 соответствуют первой ячейке листа.

Примеры проверок Excel

Примеры assertions

```
Assertions.assertTrue(file.exists());
Assertions.assertTrue(file.length() > 0);

Assertions.assertEquals(1, ExcelUtils.getSheetCount(file));
Assertions.assertTrue(ExcelUtils.getText(file).contains("admin"));
Assertions.assertEquals(
        "admin@example.com",
        ExcelUtils.getCellValue(file, "Users", 1, 2)
);
```

6. Загрузка файла через Selenide

Для загрузки файла используется элемент <input type="file">. В Selenide файл можно передать методом uploadFile(...). Если на странице есть красивая кнопка «Загрузить», сам input иногда скрыт - в Page Object нужно находить именно файловый input.

Рекомендуемое расположение тестового файла

Структура проекта

```
src/
└── test/
    └── resources/
        └── files/
            └── avatar.png
```

Пример Page Object

Идея метода загрузки

```
public class ProfilePage {

    private final SelenideElement avatarInput =
            $("input[type='file']");

    @Step("Загрузить новую аватарку")
    public ProfilePage uploadAvatar(File file) {
        avatarInput.uploadFile(file);
        return this;
    }
}
```

Это учебный пример. Локатор и дополнительные действия (сохранение, подтверждение, ожидание обновления аватарки) ученики должны адаптировать под фактическую верстку тестового сайта.

7. Домашнее задание

Цель ДЗ

Самостоятельно повторить работу с загрузкой и скачиванием файлов и написать отдельный UI-тест на изменение аватарки пользователя.

Часть 1. Повторить материал урока

1. Добавить в проект зависимости pdf-test и xls-test.
2. Переписать PdfUtils самостоятельно и разобраться, зачем нужен каждый метод.
3. Переписать ExcelUtils самостоятельно и разобраться с Workbook / Sheet / Row / Cell.
4. Скачать PDF с тестового сайта и выполнить минимум 2 содержательные проверки.
5. Скачать Excel с тестового сайта и выполнить минимум 2 содержательные проверки.
6. Не ограничиваться только Assertions.assertNotNull(file).

Часть 2. Тест-кейс: смена аватарки пользователя

Шаг	Действие	Ожидаемый результат
1	Открыть тестовый сайт.	Страница доступна.
2	Авторизоваться валидным пользователем.	Авторизация успешна.
3	Перейти в профиль пользователя.	Открыта страница профиля.
4	Выбрать изменение аватарки.	Доступен элемент загрузки файла.
5	Загрузить изображение из src/test/resources/files.	Файл принят интерфейсом.
6	Сохранить изменения, если это требуется интерфейсом.	Изменения сохранены.
7	Проверить результат.	Новая аватарка отображается у пользователя.

Требования к реализации ДЗ

- Авторизацию вынести в LoginPage.
- Работу с профилем вынести в ProfilePage.
- Файл аватарки хранить в test resources, а не использовать абсолютный путь с компьютера.
- Добавить Allure @Step для ключевых пользовательских действий.
- В тесте проверять конечный результат - аватарка действительно изменилась.
- Не помещать Selenium/Selenide-локаторы непосредственно в тестовый метод.

Что сдаем

- код теста скачивания и проверки PDF;
- код теста скачивания и проверки Excel;
- PdfUtils и ExcelUtils;
- Page Object для профиля;
- тест на смену аватарки;
- тестовые файлы, необходимые для запуска, в src/test/resources/files.

8. Чек-лист перед сдачей

- Проект собирается без ошибок.
- Тесты запускаются независимо друг от друга.
- Нет абсолютных путей вида C:\Users\... в тестовом коде.
- Скачанный PDF проверяется по содержимому.
- Скачанный Excel проверяется по содержимому.
- Загрузка аватарки выполняется через Page Object.
- Есть assertion конечного результата.
- Названия методов отражают действие или проверку.

> **Критерий готовности**
>
> Ученик должен уметь объяснить, где заканчивается ответственность Page Object и где начинается ответственность PdfUtils / ExcelUtils. Простое копирование кода без понимания не считается выполнением задания.